



IIC1001 — Algoritmos y Sistemas Computacionales — 2026-1
Interrogación 1 - Solución

Martes 14-Abril-2026

Duración: 120 minutos

Instrucciones generales

- Puntaje: 60 pts.
- El desarrollo es estrictamente individual.
- No está autorizado el uso de dispositivos electrónicos de ningún tipo.
- **Responder CADA PREGUNTA EN UNA HOJA SEPARADA. Se descontará 0.1 por no seguir esto.**
- La notación $0x$ indica que los dígitos que siguen son hexadecimales. La notación $0b$ indica que los dígitos que siguen son binarios. Ambas notaciones no son parte del número.
- Para cada ejercicio se recomienda explicitar el procedimiento utilizado. En caso que el resultado esté incorrecto pero haya procedimiento, podría obtener puntaje parcial.

1. [18p] PREGUNTA 1. Conversión de bases numéricas.

- 1.1) [9p] Complete la siguiente tabla de equivalencia entre base numéricas decimal, binario y hexadecimal. Todos los números son positivos.

binario	decimal	hexadecimal
1011		
110101		
111010011011		
	13	
	58	
	1021	
		9
		75
		A9C

R.

- Para 1011_2 , la conversión a decimal es $2^3 + 2^1 + 2^0 = 8 + 2 + 1 = 11_{10}$, y a su vez corresponde en hexadecimal a B_{16} .
- Para 110101_2 , la conversión a decimal es $2^5 + 2^4 + 2^2 + 2^0 = 32 + 16 + 4 + 1 = 53_{10}$. A su vez, si agrupamos de a 4 bit, tenemos que $110101_2 = 0011\ 0101 = 35_{16}$.
- Para 111010011011_2 , la conversión a decimal es $2^{11} + 2^{10} + 2^9 + 2^7 + 2^4 + 2^3 + 2^1 + 2^0 = 2048 + 1024 + 512 + 128 + 16 + 8 + 2 + 1 = 3739_{10}$. Si agrupamos de a 4 bit, tenemos que $111010011011_2 = 1110\ 1001\ 1011 = E9B_{16}$.
- Para 13_{10} , el equivalente hexadecimal es D_{16} , y en binario es 1101 .
- Para 58_{10} , se puede calcular el binario dividiendo sucesivamente por dos. $58 \div 2 = 29$, resto = $\boxed{0}$; $29 \div 2 = 14$, resto = $\boxed{1}$; $14 \div 2 = 7$, resto = $\boxed{0}$; $7 \div 2 = 3$, resto = $\boxed{1}$; $3 \div 2 = 1$, resto = $\boxed{1}$; $1 \div 2 = 0$, resto = $\boxed{1}$. El valor en binario es 111010_2 . Agrupando de a 4 bit podemos encontrar el hexadecimal: $111010_2 = 0011\ 1010 = 3A_{16}$.
- Para 1021_{10} , podemos notar que $1024_{10} = 1000000000_2$. Si retrocedemos tres obtenemos $1021_{10} = 111111101_2$. Alternativamente, podemos dividir por dos: $1021 \div 2 = 510$, resto = $\boxed{1}$; $510 \div 2 = 255$, resto = $\boxed{0}$; $255 \div 2 = 127$, resto = $\boxed{1}$; $127 \div 2 = 63$, resto = $\boxed{1}$; $63 \div 2 = 31$, resto =

$\boxed{1}$, $31 \div 2 = 15$, resto = $\boxed{1}$, $15 \div 2 = 7$, resto = $\boxed{1}$, $7 \div 2 = 3$, resto = $\boxed{1}$, $3 \div 2 = 1$, resto = $\boxed{1}$, $1 \div 2 = 0$, resto = $\boxed{1}$. Por lo tanto el valor en binario es 1111111101_2 . Agrupando de 4 bit obtenemos $0011\ 1111\ 1101 = 3FD$.

- Para 9_{16} , el valor decimal es 9_{10} , y en binario es 1001_2 .
- Para 75_{16} , el valor binario es $0111\ 0101 = 1110101_2$. Para el valor decimal podemos calcular a partir del binario: $1110101_2 = 2^6 + 2^5 + 2^4 + 2^2 + 2^0 = 117_{10}$. También podríamos haber calculado que $75_{16} = 7 \times 16 + 5 = 112 + 5 = 117_{10}$.
- Para $A9C_{16}$, el valor binario es $1010\ 1001\ 1100 = 101010011100_2$. Para el valor decimal podemos calcular que $A9C_{16} = 10 \times 256 + 9 \times 16 + 12 = 2560 + 144 + 12 = 2716_{10}$.

La tabla completa es:

binario	decimal	hexadecimal
1011	11	B
110101	53	35
111010011011	3739	E9B
1101	13	D
111010	58	3A
1111111101	1021	3FD
1001	9	9
1110101	117	75
101010011100	2716	A9C

0,5p por cada resultado correcto. No es necesario indicar el procedimiento en cada caso. Si hay procedimiento y se identifica un error menor, por ejemplo, un error de un bit propagado, puede asignarse 0,25p. Si el error es en el mecanismo o en el punto de partida del procedimiento, 0p.

- 1.2) [6p] Complete la siguiente tabla indicando el valor decimal correspondiente de acuerdo a la interpretación del número binario.

x	binario	valor x bit sin signo	valor x bit con signo, complemento a dos
4	0011		
4	1100		
5	01100		
5	10000		
8	00101101		
8	11100110		

R.

- Para 0011_2 con $x = 4$ bit. Si lo interpretamos como un número sin signo (positivo), la conversión es $0011_2 = 2^1 + 2^0 = 2 + 1 = 3_{10}$.
Si lo interpretamos como un número de 4 bit con signo y complemento a dos, observamos que es un número positivo (el bit de la izquierda es 0). Por lo tanto la conversión es $0011_2 = 2^1 + 2^0 = 2 + 1 = 3_{10}$.
- Para 1100_2 con $x = 4$ bit. Si lo interpretamos como un número sin signo, la conversión es $1100_2 = 2^3 + 2^2 = 8 + 4 = 12_{10}$.
Si lo interpretamos como un número de 4 bit con signo y complemento a dos, observamos que es un número negativo (el bit de la izquierda es 1). Para conocer la magnitud, hacemos la conversión: $1100 \rightarrow 0011 \xrightarrow{+1} 0100 = 4_{10}$. Por lo tanto el valor correspondiente a 1100_2 es -4_{10} .
- Para 01100_2 con $x = 5$ bit. Si lo interpretamos como un número sin signo (positivo), la conversión es $01100_2 = 2^3 + 2^2 = 8 + 4 = 12_{10}$.
Si lo interpretamos como un número de 5 bit con signo y complemento a dos, observamos que es un número positivo. La conversión es $0100_2 = 2^3 + 2^2 = 8 + 4 = 12_{10}$.
- Para 10000_2 con $x = 5$ bit. Si lo interpretamos como un número sin signo, la conversión es $10000_2 = 2^4 = 16_{10}$.
Si lo interpretamos como un número de 5 bit con signo y complemento a dos, observamos que es un número negativo (empieza con 1). Para conocer la magnitud, hacemos la conversión $10000 \rightarrow 01111 \xrightarrow{+1} 10000 = 16_{10}$. Por lo tanto el valor correspondiente a 10000_2 es -16_{10} .
- Para 00101101_2 con $x = 8$ bit. Si lo interpretamos como un número sin signo, la conversión es $00101101_2 = 2^5 + 2^3 + 2^2 + 2^0 = 45_{10}$.
Si lo interpretamos como un número de 8 bit con signo y complemento a dos, observamos que es un número positivo (empieza con 0). Por lo tanto, la conversión es directa: $00101101_2 = 2^5 + 2^3 + 2^2 + 2^0 = 45_{10}$.
- Para 11100110_2 con $x = 8$ bit.
Si lo interpretamos como un número sin signo, la conversión es $11100110 = 2^7 + 2^6 + 2^5 + 2^2 + 2^1 = 128 + 64 + 32 + 4 + 2 = 230_{10}$.
Si lo interpretamos como un número con signo y complemento a dos, observamos que es un número negativo (empieza con 1). Para conocer la magnitud hacemos la conversión: $11100110 \rightarrow 00011001 \xrightarrow{+1} 00011010 = 2^4 + 2^3 + 2^1 = 16 + 8 + 2 = 26$. Por lo tanto, el valor correspondiente a 11100110 es -26_{10} .

La tabla completa es:

x	binario	valor x bit sin signo	valor x bit con signo, complemento a dos
4	0011	3	3
4	1100	12	-4
5	01100	12	12
5	10000	16	-16
8	00101101	45	45
8	11100110	234	-26

0,5p por cada valor correcto. No es necesario indicar el procedimiento en cada caso. Si hay procedimiento

y se identifica un error menor, por ejemplo, un error de un bit propagado, puede asignarse 0,25p. Si el error es en el mecanismo o en la interpretación o en el punto de partida del procedimiento, 0p.

- 1.3) **[3p]** ¿Cuál es el rango de valores enteros que se puede representar usando 3 bit con signo y complemento a dos?

R.

Usando el formato de $n = 3$ bit con signo y complemento a dos, el rango de valores va desde -2^{n-1} hasta $2^{n-1} - 1$, que es desde -4 hasta 3 .

1,5p por valor inicial correcto

1,5p por valor final correcto

Si bien esto no se pedía, se puede construir la tabla de conversión para números de 3 bit con signo y complemento a dos, donde se puede verificar que el menor valor es -4 y el mayor valor 3 .

Bin	Dec
000	0
001	+1
010	+2
011	+3
100	-4
101	-3
110	-2
111	-1

2. [16p] PREGUNTA 2. Operaciones con números binarios.

2.1) [8p] Calcule las siguientes sumas de números. **Todos** los números usan una representación 8 bit con signo y complemento a dos. Indique el resultado correspondiente en base 10 con signo. Si ocurre un *overflow*, indíquelo explícitamente.

a) $0b00110101 + 0b00011010$

R.

$$\begin{array}{r} \textcolor{red}{11} \\ 00110101 \\ + 00011010 \\ \hline 01001111 \end{array}$$

El resultado es $01001111_2 = 79_{10}$. Se trata de una suma de dos positivos que dan un resultado positivo, por lo tanto no hay *overflow*. A modo de comprobación, $00110101_2 = 53_{10}$, $00011010_2 = 26_{10}$, por lo que el resultado debería ser $79_{10} = 01001111_2$.

b) $0b01101111 + 0b00010101$

R.

$$\begin{array}{r} \textcolor{red}{11111111} \\ 01101111 \\ + 00010101 \\ \hline 10000100 \end{array}$$

El resultado es 10000100_2 , que es un número negativo, y cuya magnitud es $10000100 \rightarrow 01111011 \xrightarrow{+1} 01111100 = 124_{10}$. Por lo tanto $10000100_2 = -124_{10}$. Se trata de una suma de dos positivos que da un resultado negativo, por lo tanto hay un *overflow*.

A modo de comprobación, la suma corresponde a $01101111 = 111_{10}$, y $00010101 = 21_{10}$, cuya suma es 132_{10} . Este número cae fuera del rango representable con 8 bit, que es $[-128, 127]$.

c) $0x5A + 0xA6$

R.

$$\begin{array}{r} \textcolor{red}{111111} \\ 01011010 \\ + 10100110 \\ \hline \cancel{1}00000000 \end{array}$$

El *carry* final se descarta. Se trata de la suma de un positivo y un negativo, por lo tanto no ocurre *overflow*. El resultado es $00000000_2 = 0_{10}$.

A modo de comprobación, la suma es entre $01011010_2 = 90_{10}$, y 10100110_2 que es un negativo cuya magnitud es $10100110 \rightarrow 01011001 \xrightarrow{+1} 01011010 = 90_{10}$. Por lo tanto $10100110_2 = -90_{10}$. Se comprueba que el resultado debería ser 0_{10} .

d) $0x7C + 0x0A$

R.

$$\begin{array}{r} \textcolor{red}{1111} \\ 01111100 \\ + 00001010 \\ \hline 10000110 \end{array}$$

Se trata de una suma de dos positivos que entrega como resultado un negativo, por lo tanto se produce *overflow*. El resultado obtenido corresponde a 10000110 que es un negativo cuya magnitud es $10000110 \rightarrow 01111001 \xrightarrow{+1} 01111010 = 122_{10}$. Por lo tanto $10000110_2 = -122_{10}$.

A modo de comprobación, la suma corresponde a $01111100_2 = 124_{10}$, y $00001010_2 = 10_{10}$ cuyo resultado debería ser 134. Este número cae fuera del rango representable con 8 bit, que es $[-128, 127]$.

Para todos los casos, 2p cada uno.

1p. Resultado correcto en decimal. El resultado debe ser el que corresponde a la interpretación de 8 bit con signo. Por ejemplo, en el (b) el resultado esperado es -124_{10} , y **no** 132_{10} que sería la interpretación sin signo. Si hay un error menor, 0,5p

1p. Por indicar si hay o no *overflow*. Si no indica nada se puede suponer que es ausencia de *overflow*.

2.2) [8p] Calcule las siguientes restas de números. **Todos** los números usan la representación 8 bit con signo y complemento a dos. Indique el resultado correspondiente en base 10 con signo. Si ocurre un *overflow*, indíquelo explícitamente.

a) $0b00110110 - 0b00010101$

R.

Como se trata de una resta, se calcula la suma con el valor negativo de $00010101 \rightarrow 11101010 \xrightarrow{+1} 11101011$.

$$\begin{array}{r} \textcolor{red}{111111} \\ 00110110 \\ + 11101011 \\ \hline \cancel{1}00100001 \end{array}$$

El *carry* final se descarta. Se está sumando un positivo y un negativo, por lo tanto no hay *overflow*. El resultado es $00100001_2 = 33_{10}$.

A modo de comprobación, la resta es entre $00110110_2 = 54_{10}$ y $00010101_2 = 21_{10}$, cuyo resultado es 33_{10} .

b) $0b01001000 - 0b00111100$

R.

Como se trata de una resta, se calcula la suma con el valor negativo de $00111100 \rightarrow 11000011 \xrightarrow{+1} 11000100$.

$$\begin{array}{r} \textcolor{red}{11} \\ 01001000 \\ + 11000100 \\ \hline \cancel{1}00001100 \end{array}$$

El *carry* final se descarta. Se está sumando un positivo y un negativo, por lo tanto no hay *overflow*. El resultado es $00001100_2 = 12_{10}$.

A modo de comprobación, la resta es entre $01001000_2 = 72_{10}$ y $00111100_2 = 60_{10}$, cuyo resultado es 12_{10} .

c) $0x2C - 0x56$

R.

Como se trata de una resta, se calcula la suma con el valor negativo de $56_{16} = 01010110 \rightarrow 10101001 \xrightarrow{+1} 10101010$.

$$\begin{array}{r} \textcolor{red}{1} \textcolor{red}{1} \\ 00101100 \\ + 10101010 \\ \hline 11010110 \end{array}$$

Se está sumando un positivo y un negativo, por lo tanto no hay *overflow*. El resultado es 11010110_2 que es un negativo y su magnitud es $11010110 \rightarrow 00101001 \xrightarrow{+1} 00101010$. Por lo tanto $11010110_2 = -42_{10}$.

A modo de comprobación, la resta es entre $00101100_2 = 44_{10}$ y $01010110_2 = 86_{10}$, cuyo resultado es -42_{10} .

d) $0x90 - 0x60$

R.

Como se trata de una resta, se calcula la suma con el valor negativo de $60_{16} = 01100000 \rightarrow 10011111 \xrightarrow{+1} 10100000$.

$$\begin{array}{r}
 \textcolor{red}{1} \\
 10010000 \\
 + 10100000 \\
 \hline
 \cancel{1}00110000
 \end{array}$$

El *carry* final se descarta. Se están sumando dos negativos, y el resultado da positivo. Por lo tanto se produce *overflow*. El resultado es $00110000_2 = 48_{10}$.

A modo de comprobación, la resta es entre $10010000_2 = -112_{10}$ y $01100000_2 = 96_{10}$, cuyo resultado es -208_{10} . Este número cae fuera del rango representable con 8 bit, que es $[-128, 127]$.

Para todos los casos, 2p cada uno.

1p. Resultado correcto en decimal. El resultado debe ser el que corresponde a la interpretación de 8 bit con signo. Por ejemplo, en el (d) el resultado esperado es 48_{10} , y **no** -208_{10} que sería la interpretación sin signo. Si hay un error menor, 0,5p

1p. Por indicar si hay o no *overflow*. Si no indica nada se puede suponer que es ausencia de *overflow*.

3. [12p] PREGUNTA 3. Codificación

- 3.1) [9p] Se propone un algoritmo de encriptación basado en codificación ASCII y sumas. A partir de un *string* m en ASCII, los pasos son: (1) convertir m a bytes (hexadecimal), (2) sumar k a cada byte del *string*, (3) convertir el resultado a ASCII.

Para los siguientes strings m , y valores de k , muestra el resultado de cada paso del algoritmo.

- a) $m = \text{DCContainer}$ $k = 0x04$

R.

Conversión a bytes:

0x44 0x43 0x43 0x6F 0x6E 0x74 0x61 0x69 0x6E 0x65 0x72

Sumar 0x04:

0x48 0x47 0x47 0x73 0x72 0x78 0x65 0x6D 0x72 0x69 0x76

Convertir a ASCII:

HGGsrxmriV

- b) $m = \text{ABRIL}$ $k = 0x20$

R. Conversión a bytes:

0x41 0x42 0x52 0x49 0x4C

Sumar 0x20

0x61 0x62 0x72 0x69 0x6C

Convertir a ASCII:

abril

- c) $m = [\text{gZj}] \text{dgY}, \% ; \text{gdg} ($ $k = 0x08$

R.

Conversión a bytes:

0x5B 0x67 0x5A 0x6A 0x5D 0x64 0x67 0x59 0x2C 0x25 0x3B 0x67 0x64 0x67
0x28

Sumar 0x08

0x63 0x6F 0x62 0x72 0x65 0x6C 0x6F 0x61 0x34 0x2D 0x43 0x6F 0x6C 0x6F
0x30

Convertir a ASCII:

cobreloa4-Colo0

Para cada uno, 3p.

1p por cada paso.

0,5 si hay 1 o 2 errores en cada paso. Descontar solo una vez.

- 3.2) **[3p]** La notación ASCII, que usa 7-bit, es limitada en la cantidad de caracteres (128) que puede representar. Codificaciones como UTF-8 amplían ese rango usando entre 1 y 4 bytes para representar millones de caracteres. Al leer UTF-8, ¿cómo sabe el sistema si la representación es de 1, 2, 3 ó 4 byte?

R.

UTF-8 utiliza los primeros bit de cada byte que lee. Si el primer byte que lee empieza con 0, significa que es un caracter de 1 byte (ASCII). Si el primer byte empieza con 110 entonces se trata de una representación de 2 byte. Si empieza con 1110, es una representación de 3 byte. Si empieza con 11110, es una representación de 4 byte. Adicionalmente, cada uno de los 1, 2 ó 3 byte siguientes debe empezar con 10.

2p. Por indicar que depende de los bits (o de una parte) del primer byte. No es necesario mencionar las secuencias exactas. 1p si hay algún error menor en la descripción.

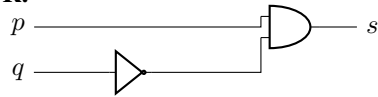
1p. Por mencionar que los siguientes bytes empiezan de una manera particular (10), aunque no es necesario mencionar la secuencia exacta.

4. [14p] PREGUNTA 4. Circuitos digitales

4.1) [4p] Para las siguientes fórmulas proposicionales construya un circuito digital equivalente:

a) $s = p \wedge \neg q$

R.



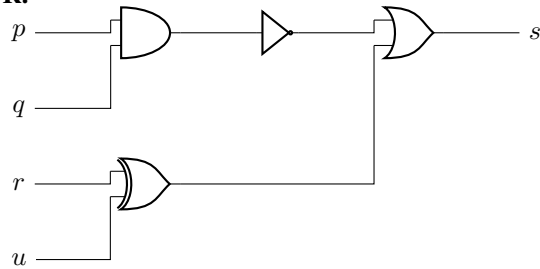
2p.

1p por ambas celdas correctas (0,5 cada una).

1p por las conexiones entre las variables.

b) $s = \neg(p \wedge q) \vee (r \oplus u)$

R.

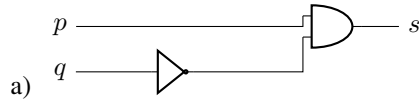


2p por cada uno.

1p por cada mitad de la proposición representada correctamente. Descuento de 0,25p por cada compuerta incorrecta.

1 por conexiones correctas.

4.2) [4p] Para los siguientes circuitos digitales construya las tablas de salida correspondientes



R.

p	q	$\neg q$	$s = p \wedge \neg q$
0	0	1	0
0	1	0	0
1	0	1	1
1	1	0	0

Como observación, este circuito solo entrega 1 “cuando p es 1 y q no es 1”, lo que se condice con la expresión “ p y no q ”.

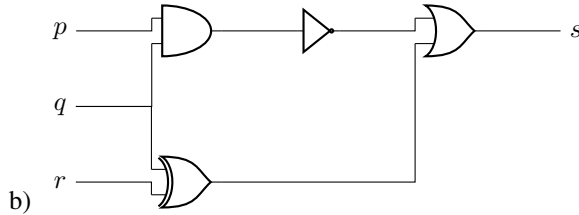
2p.

0,5p por cada fila correcta.

Es suficiente con las columnas de p , q y s . No es necesario que estén las columnas intermedias, pero si están pueden ayudar a indicar errores. Tampoco es necesario que escriban la expresión proposicional, si bien es un buen paso hacerlo.

Los valores deben estar como 0 y 1. Si usan V y F , o “Verdadero” y “Falso”, o similar, asignar máximo 1p al ítem.

El orden de las filas no es importante. Da lo mismo si empiezan con 0, 0 o con 1, 1 mientras los valores de cada fila sean correctos.



R.

p	q	r	$p \wedge q$	$\neg(p \wedge q)$	$q \oplus r$	$s = \neg(p \wedge q) \vee (q \oplus r)$
0	0	0	0	1	0	1
0	0	1	0	1	1	1
0	1	0	0	1	1	1
0	1	1	0	1	0	1
1	0	0	0	1	0	1
1	0	1	0	1	1	1
1	1	0	1	0	1	1
1	1	1	1	0	0	0

Este circuito entrega 0 cuando hay al menos un 0 en las entradas. Es posible demostrar que esta expresión es equivalente a $\neg p \vee \neg q \vee \neg r$ (pero no era el objetivo de la pregunta).

2p.

−0,5p por cada fila incorrecta (mínimo 0).

Es suficiente con las columnas de p , q , r y s . No es necesario que estén las columnas intermedias, pero si están pueden ayudar a indicar errores. Tampoco es necesario que escriban la expresión proposicional, si bien es un buen paso hacerlo.

Los valores deben estar como 0 y 1. Si usan V y F , o “Verdadero” y “Falso”, o similar, asignar máximo 1p al ítem.

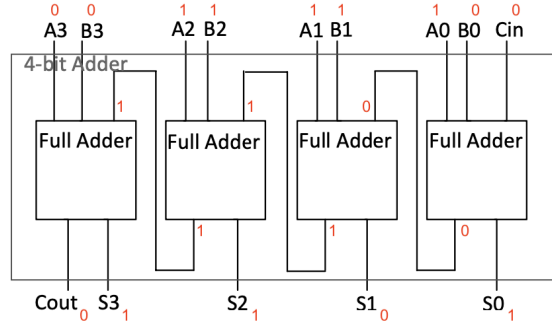
El orden de las filas no es importante. Da lo mismo si empiezan con 0, 0, 0 o con 1, 1, 1 mientras los valores de cada fila sean correctos.

4.3) [6p] Usando el circuito sumador de 4-bit, indique las entradas y salidas de cada *full adder* y del sumador completo para las siguientes sumas binarias:

a) 0111 + 0110

R.

Para este caso, $A = A_3A_2A_1A_0 = 0111$, y $B = B_3B_2B_1B_0 = 0110$. Además $C_{in} = 0$.

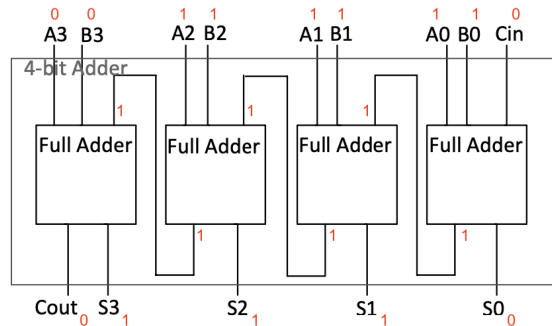


El resultado es $C_{out} = 0$, y $S = S_3S_2S_1S_0 = 1101$.

b) 0111 + 0111

R.

Para este caso, $A = A_3A_2A_1A_0 = 0111$, y $B = B_3B_2B_1B_0 = 0111$. Además $C_{in} = 0$.

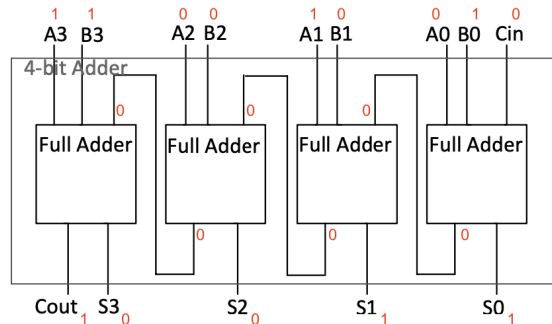


El resultado es $C_{out} = 0$, y $S = S_3S_2S_1S_0 = 1110$.

c) 1010 + 1001

R.

Para este caso, $A = A_3A_2A_1A_0 = 1010$, y $B = B_3B_2B_1B_0 = 1001$. Además $C_{in} = 0$.



El resultado es $C_{out} = 1$, y $S = S_3S_2S_1S_0 = 0011$.

Para cada uno, 2p.

1p por obtener el valor correcto, incluyendo el C_{out} y los bit de S . Si incluyen el valor del C_{out} dentro de S es un error menor ($-0,5p$).

0,5p por dibujar el sumador de 4-bit usando 4 *Full Adder* y por agregar sus entradas y salidas. No es necesario dibujar las celdas internas de cada *Full Adder*.

0,5p por indicar los valores correctos en cada bit de A , de B , S y C_{out} . Basta con indicar los bit una vez en el diagrama. No es necesario replicarlos en cada extremo de las conexiones.
